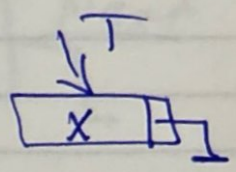
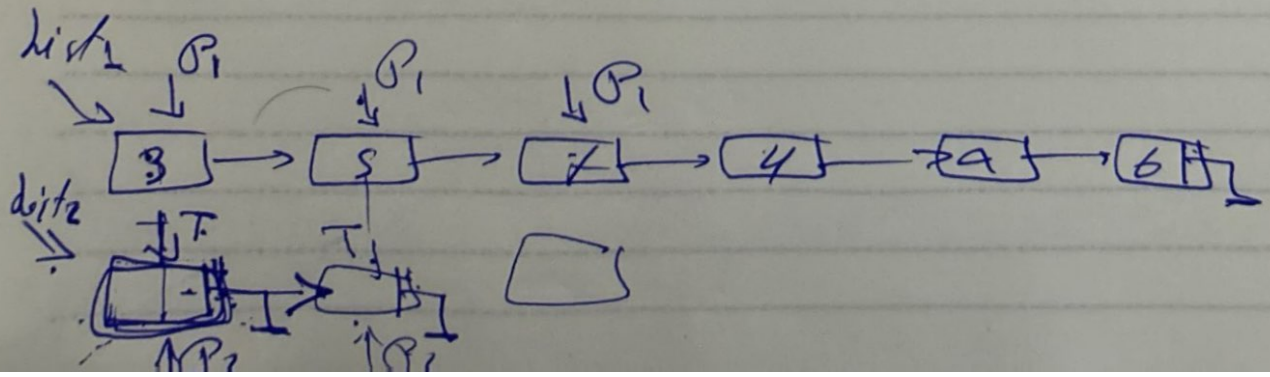
20/3/2025

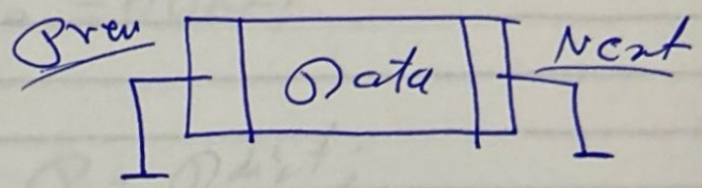
```
void COPY ( Node* list1 , Node* list2)
{
    Node* P1 = list1;
    while( P1 != NULL)
    {
        Node* T = new Node();
        T->Data = P1->Data;
        T->Next = NULL;
        if (list2 == NULL)
        {
            list2 = T;
            Node* P2 = list2;
        }
        else
        {
            P2->Next = T;
            P2 = P2->Next;
        }
        P1 = P1->Next;
    }
```

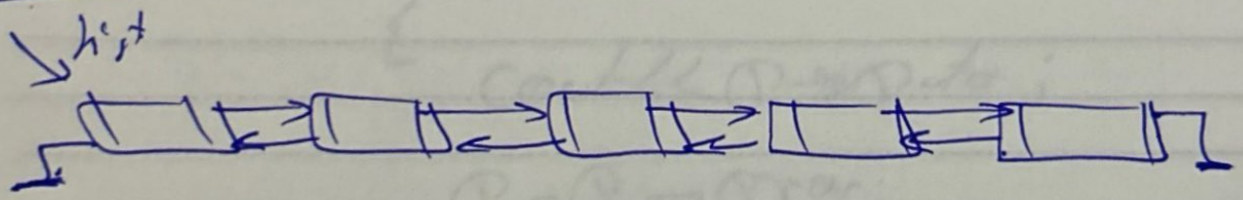




Double linked list

```
struct Node
{
  int Data;
  struct Node * Next;
  struct Node * Prev;
};
```





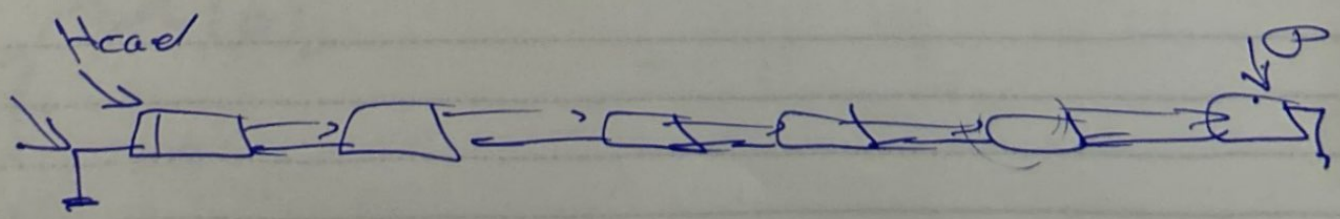
Sum count search Avg SumEv

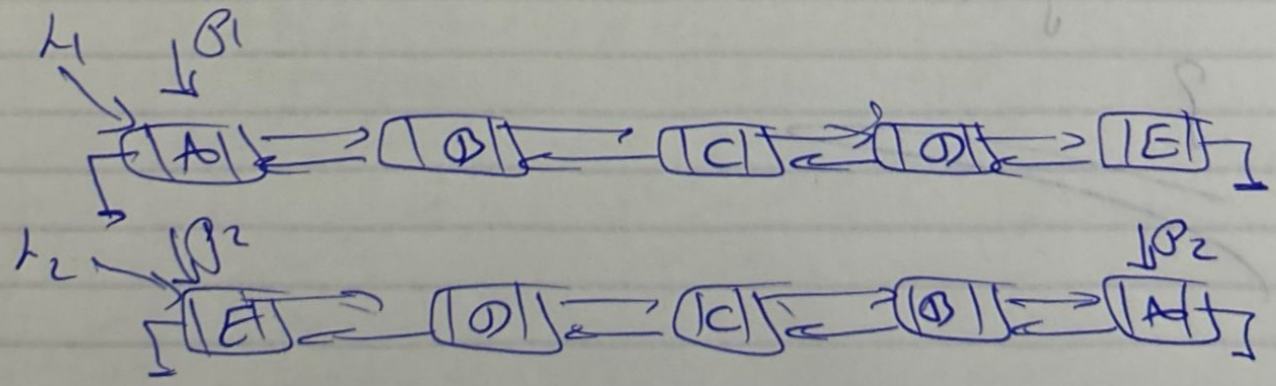
Max . .

Insert
delete

```
void PrintReverse( Node *Head)
{
        if (Head != NULL)
        {
            Node * P = Head;
            while (P -> Next != NULL)
                P = P -> Next;

            while (P != NULL)
            {
                cout << P -> Data;
                P = P -> Prev;
            }
        }
}
```

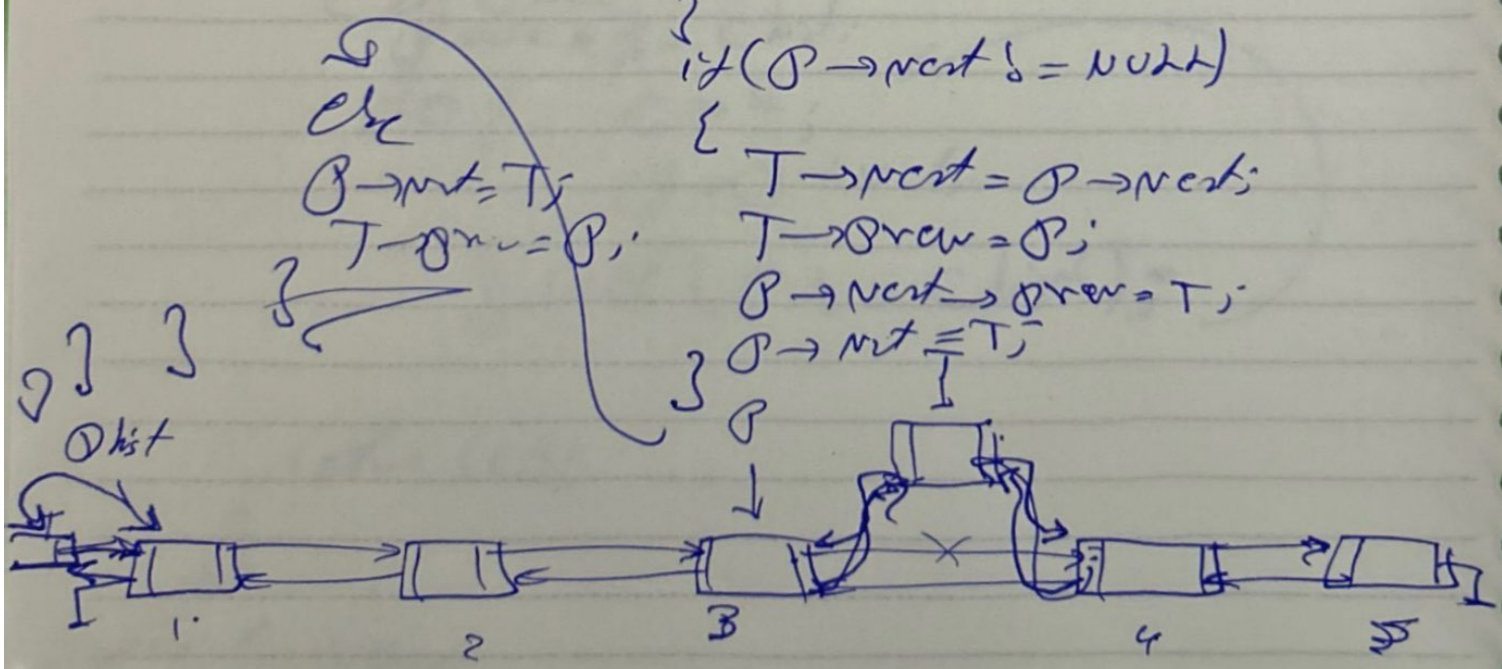



h_1
β_1
A
B
C
D
E
h_2
β_2
E
D
C
B
A
β_2

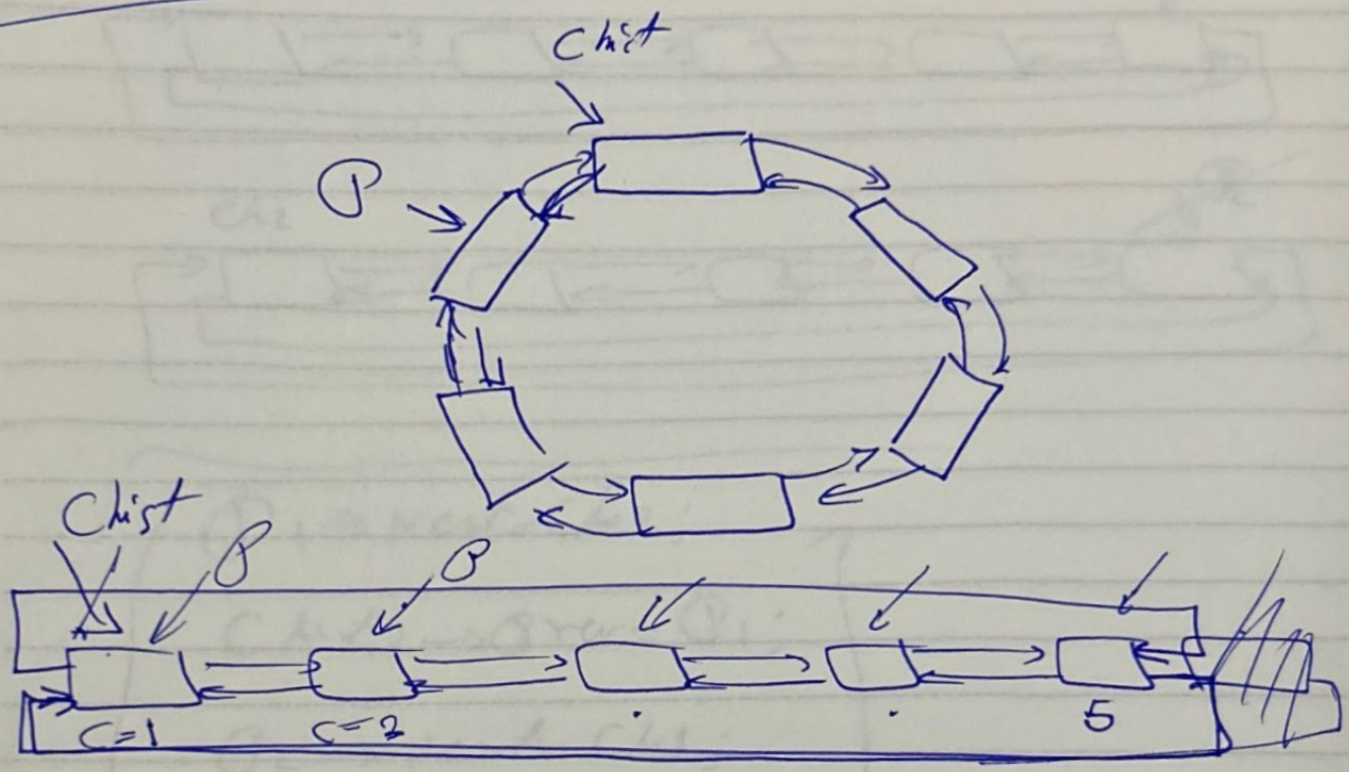
```
int Reverse(Node* L1, Node* L2)
{ Node* P1 = L1;
  Node* P2 = L2;
  if(P2 != NULL)
  {  while(P2->Next != NULL)
          P2 = P2->Next;
  }
  while(P1 != NULL && P2 != NULL)
  { if(P1->Data != P2->Data)
          return(0);
    P1 = P1->Next;
    P2 = P2->Prev;
  }
  if(P1 == NULL && P2 == NULL)
          return(1);
  return(0);
}
```

```
void InsertK ( Node *Head, int x, int K)
{
        Node * T = new Node( ),
        T -> Data = x;
        T -> Next = T -> Prev = NULL;

        if (Head == NULL)
            Head = T;
        else
        { if (K == 1)
            {
              T -> Next = Head;
              Head -> Prev = T;
              Head = T;
            }
            else{ int C = 1;
                  Node * P = Head;
                  while (P -> Next != NULL && C < (K-1))
                  { C ++;
                    P = P -> Next
                  }
                  if (P -> Next != NULL)
                  {
                    T -> Next = P -> Next;
                    T -> Prev = P;
                    P -> Next -> Prev = T;
                    P -> Next = T;
                  }
                  else
                  P -> Next = T;
                  T -> Prev = P;
                  }
        }
}
```

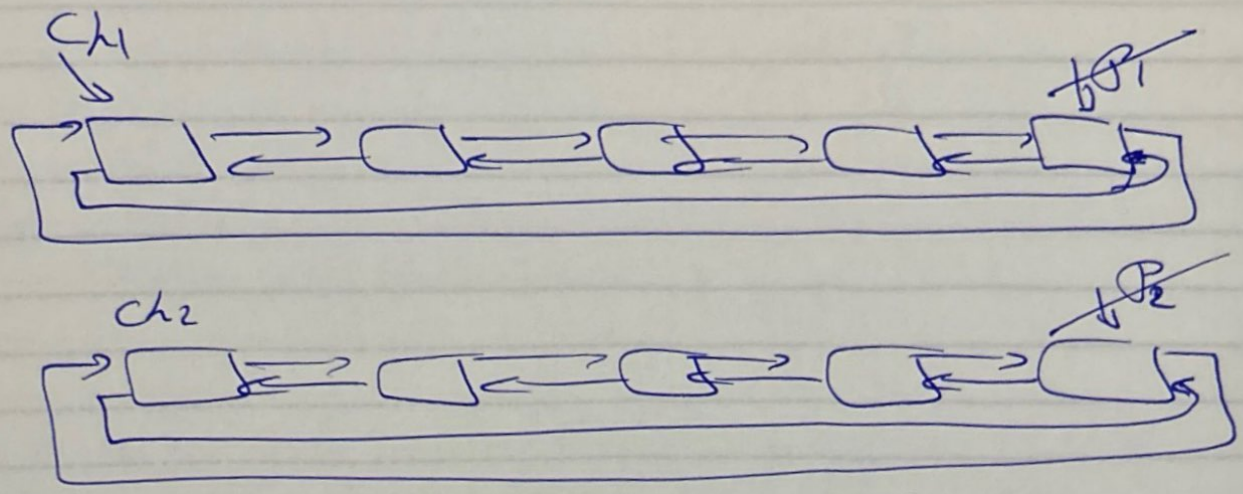


Circular linked list




```
int count( Node *Chist)
{
    if(Chist != NULL)
    {  int C=0;
       Node *P= Chist;
       do{   C++;
             P=P->Next;
       } while (P != Chist);

    return (C);
    }
return(0);
}
```



$P_1 \rightarrow next = Ch2;$

$Ch2 \rightarrow prev = P_1;$

$P_2 \rightarrow next = Ch1;$

$Ch1 \rightarrow prev = P_2.$